

```

1  (*
2                                     CS 51 Lab 19
3                                     Greenberg-Hastings Cellular Automaton
4                                     https://en.wikipedia.org/wiki/Greenberg_Hastings_cellular_automaton
5
6  Implementation of the Greenberg-Hastings model described in the
7  article
8
9      Greenberg, J. M., and S. P. Hastings. "Spatial Patterns for
10     Discrete Models of Diffusion in Excitable Media." SIAM Journal on
11     Applied Mathematics 34, no. 3 (1978): 515-23.
12     http://www.jstor.org/stable/2100950.
13 *)
14
15  (*
16     *****
17     WARNING: If you have photosensitive epilepsy, you should avoid
18     viewing the Greenberg-Hastings cellular automaton. It exhibits
19     rapidly flashing visual patterns.
20     *****
21  *)
22
23  module G = Graphics ;;
24
25  (* Automaton parameters *)
26  let cGRID_SIZE = 100 ;;          (* width and height of grid in cells *)
27  let cSPARSITY = 25 ;;           (* inverse of proportion of cells initially live *)
28
29  (* Rendering parameters *)
30  let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
31  let cSIDE = 8 ;;                (* width and height of cells in pixels *)
32  let cRENDER_FREQUENCY = 1      (* how frequently grid is rendered (in ticks) *) ;;
33  let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
34
35  type gh_state =
36  | Resting
37  | Excited
38  | Refractory
39
40  (* offset index off -- Returns the `index` offset by `off` allowing
41     for wraparound. *)
42  let rec offset (index : int) (off : int) : int =
43    if index + off < 0 then
44      cGRID_SIZE - (offset ~-index ~-off)

```

```

45     else
46         (index + off) mod cGRID_SIZE ;;
47
48 (* gh_update grid i j -- *)
49 let gh_update (grid : gh_state array array) (i : int) (j : int) =
50     let neighbors = ref 0 in
51     for i' = ~-1 to ~+1 do
52         for j' = ~-1 to ~+1 do
53             (* only consider the four neighbors sharing an index *)
54             if (i' = 0 || j' = 0) && not (i' = 0 && j' = 0) then
55                 let i_ind = offset i i' in
56                 let j_ind = offset j j' in
57                 neighbors := !neighbors
58                     + match grid.(i_ind).(j_ind) with
59                     | Resting -> 0
60                     | Refractory -> 0
61                     | Excited -> 1
62         done
63     done;
64     match grid.(i).(j) with
65     | Excited -> Refractory
66     | Refractory -> Resting
67     | Resting -> if !neighbors > 0 then Excited else Resting ;;
68
69 (* gh_random_state () -- Returns a random cell state. *)
70 let gh_random_state () =
71     let states = [Excited; Refractory; Resting; Resting; Resting; Resting] in
72     List.nth states (Random.int (List.length states))
73
74 module GHSpec : (Cellular.AUT_SPEC
75     with type state = gh_state) =
76     struct
77         type state = gh_state
78         let name : string = "Greenberg-Hastings"
79         let initial : state = Resting
80         let grid_size : int = cGRID_SIZE
81         let random_state = gh_random_state
82         let update = gh_update
83         let cell_color (cell : state) : G.color =
84             match cell with
85             | Resting -> G.rgb 245 240 246
86             | Excited -> G.rgb 56 95 113
87             | Refractory -> G.rgb 43 65 98
88         let side_size : int = cSIDE
89         let legend_color : G.color = cCOLOR_LEGEND
90         let font : string option = cFONT

```

```

91     let render_frequency : int = cRENDER_FREQUENCY
92 end ;;
93
94 module Aut = Cellular.Automaton (GHSpec) ;;
95
96 (*.....
97     Running the automaton and displaying the results
98 *)
99
100 (* main seed -- Runs the automaton using the provided random 'seed'
101     (for replicability), displaying updates to the graphics window. *)
102 let main seed =
103
104     Random.init seed;      (* initialize randomness *)
105     Aut.graphics_init ();  (* ... and graphics window *)
106
107     (* Initialize the grid to a simple half-line of excited and
108     refractory cells, as per the discussion in th article cited at the
109     top of the file. This generates the typical spiral artifacts
110     described in the paper. *)
111     let initial_grid = Aut.fresh_grid () in
112     for i = cGRID_SIZE / 2 to cGRID_SIZE - 1 do
113         initial_grid.(i).(cGRID_SIZE / 2) <- Refractory;
114         initial_grid.(i).(cGRID_SIZE / 2 + 1) <- Excited
115     done;
116
117     Aut.run_grid initial_grid ;;
118
119 (* Run the automaton with user-supplied seed *)
120 let _ =
121     if Array.length Sys.argv <= 1 then
122         Printf.printf "Usage: %s <seed>\n where <seed> is integer seed\n"
123             Sys.argv.(0)
124     else
125         main (int_of_string Sys.argv.(1)) ;;
126

```